CBC Digital Learning Platform

Stage E — Architectural Design

Software Architecture Document (SAD)

Version 1.0 | April 2026 | Draft — Design Phase

Prepared by: Project Team — End of Year Project
Uganda Institute of ICT (UICT)

Unified Software Architectural Process
Stage E: Architectural Design

1. Introduction

1.1 Purpose of Stage E

Stage E of the Unified Software Architectural Process — Architectural Design — builds directly on the behavioural models produced in Stage D. Where Stage D defined how objects interact and transition over time, Stage E defines where those objects and interactions live: the structural blueprint of the system, how components are assembled, how they communicate, and how they are deployed in a real-world cloud environment.

This document — the Software Architecture Document (SAD) — is the authoritative output of Stage E. It establishes the architectural decisions that will govern all subsequent stages, including database design (Stage F) and UI/UX design (Stage G).

1.2 Architectural Goals

The CBC Digital Learning Platform architecture is designed to satisfy the following goals derived directly from the SRS v1.0:

- Serve 100,000 concurrent learners nationally without degradation (NFR-S-01, NFR-S-02)
- Operate reliably on 2G/3G connectivity in rural Uganda (NFR-P-01, NFR-P-03)
- Support offline lesson access through a progressive web application strategy (NFR-O-01 to NFR-O-03)
- Ground AI tutor responses in NCDC domestic curriculum content via MCP (FR-AI-02, FR-AI-03)
- Enforce strict role-based access control and learner data privacy by default (NFR-SEC-04, NFR-SEC-05)
- Maintain 99% uptime during school hours with graceful degradation of the AI layer (NFR-A-01, NFR-A-03)

1.3 Architectural Approach: Layered Monolith

The platform adopts a Layered Monolith architecture. This decision is deliberate and appropriate for the following reasons:

- Prototype and academic scope: a monolith delivers the full system without the operational overhead of service orchestration, container registries, and inter-service networking that microservices demand.
- Single deployment unit: the entire backend deploys as one Django application, simplifying CI/CD pipelines, debugging, and academic assessment.
- Clear layer separation: the monolith is internally structured into four strict layers — Presentation, Application, AI, and Data — which enforces clean separation of concerns while keeping integration simple.
- Upgrade path: the layered structure allows the AI layer and individual application services to be extracted into standalone microservices in future phases without rewriting business logic.

The four layers are: Presentation (React SPA), Application (Django/DRF backend services), AI (AI Tutor + MCP Server), and Data (PostgreSQL, Redis, S3, IndexedDB).

2. Layer Architecture Overview

2.1 Architecture Diagram

Figure E-01 below presents the full four-layer architecture of the CBC Digital Learning Platform, showing each layer, its internal components, and the communication mechanisms between layers.

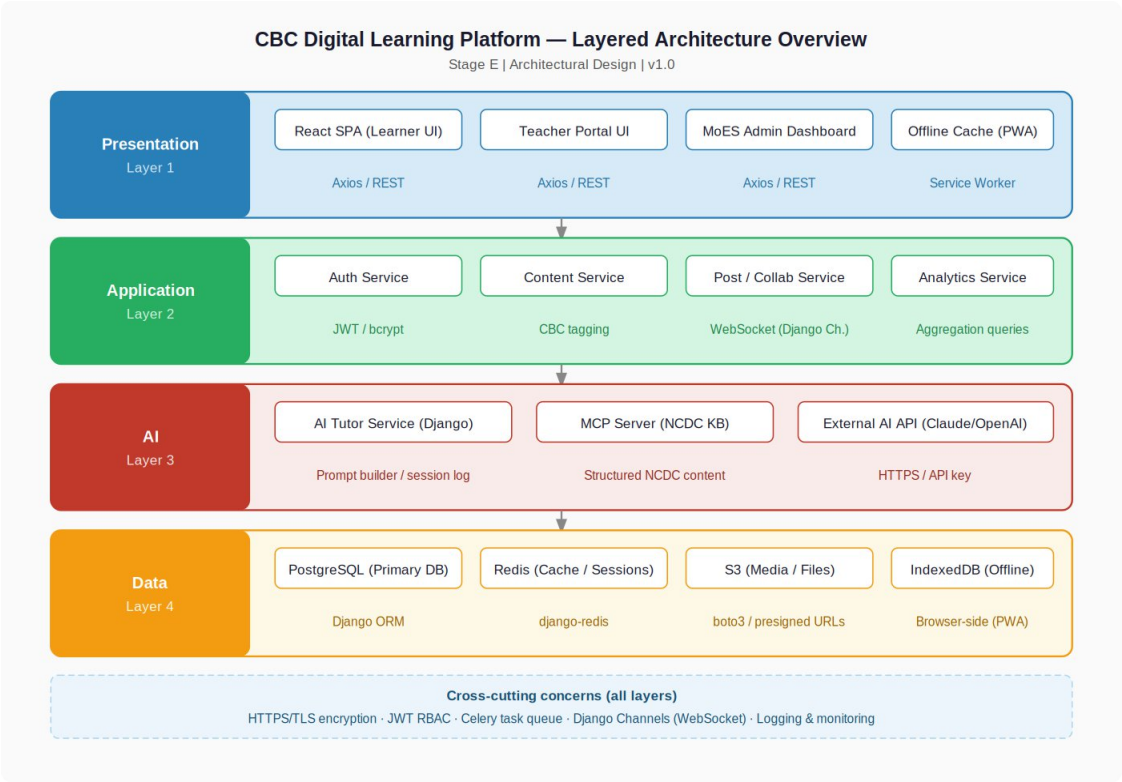

Figure E-01: CBC Digital Learning Platform — Layered Architecture Overview

2.2 Layer Descriptions

Layer 1 — Presentation Layer

The Presentation Layer is a React Single-Page Application (SPA) delivered to the browser. It comprises four functional modules: the LearnerApp (content browsing, AI tutor interface, posts and collaboration), the TeacherApp (material upload and management), the MoES/NCDC Admin Dashboard (curriculum management and analytics), and the OfflineModule (a Progressive Web Application service worker paired with browser-side IndexedDB for offline lesson access).

All modules communicate with the backend over HTTPS REST (port 443) via Axios, and over WebSocket (Django Channels) for real-time collaboration. The SPA is built as a static production build and served via AWS CloudFront CDN with origin in S3, ensuring low-latency delivery across Africa.

Layer 2 — Application Layer

The Application Layer is the Django/DRF backend. It is organised into eight logically separated Django applications, each owning a distinct domain of business logic. These are: AuthService (registration, login, JWT issuance, RBAC), ContentService (CBC lessons and teacher materials with subject/level/competency tagging), PostService (learner posts, comments, reactions), CollabService (real-time collaborative sessions via Django Channels WebSocket), ProgressService (competency attainment tracking against the CBC framework), AnalyticsService (national aggregated reporting for

MoES/NCDC), OfflineSyncService (packaging and delta-syncing lesson content for offline download), and MediaService (file upload and AWS S3 presigned URL management).

The backend runs on Gunicorn (HTTP) and Daphne (ASGI for WebSocket), behind an Nginx reverse proxy for SSL termination. Celery with Redis as broker handles asynchronous tasks such as analytics aggregation and notification dispatch.

Layer 3 — AI Layer

The AI Layer sits between the Application Layer and the external AI inference service. It consists of two components: the AITutorService and the MCP Server. The AITutorService receives a learner query from the Application Layer, retrieves relevant structured content from the NCDC knowledge base via the MCP Server, builds a context-enriched prompt, and dispatches it to the External AI API. The response is returned to the learner and logged as an AISession record in PostgreSQL.

The MCP (Model Context Protocol) Server acts as a structured query interface over the NCDC curriculum content stored in PostgreSQL. It allows the AI model to access specific CBC content — subject material, competency definitions, and learning objectives — in a controlled, structured manner, ensuring responses are grounded in Ugandan domestic academic content rather than generic foreign knowledge bases.

The external AI inference provider is Claude (Anthropic) or OpenAI, accessed over HTTPS with API key authentication. This call is outbound only and does not expose any internal platform data beyond the constructed prompt.

Layer 4 — Data Layer

The Data Layer consists of four storage components. PostgreSQL (AWS RDS, af-south-1) is the primary relational database storing all domain objects defined in Stage C: users, learner profiles, lessons, posts, AI sessions, competency progress records, and the NCDC knowledge base. Redis (AWS ElastiCache) handles session tokens, JWT blacklisting, Celery task brokering, and Django Channels pub/sub for WebSocket message routing. AWS S3 stores all binary media — uploaded teacher materials, images, and packaged offline content bundles. IndexedDB is the browser-side offline store, managed by the PWA service worker to hold pre-downloaded lesson content for connectivity-free access.

Cross-Cutting Concerns

Several concerns apply uniformly across all layers: all traffic uses HTTPS/TLS 1.2 or higher; JWT tokens enforce role-based access control at every API endpoint; Django Channels provides the WebSocket infrastructure for real-time features; Celery processes all async operations; and CloudWatch provides centralised logging and alerting.

3. Component Diagram

3.1 Component Overview

Figure E-02 presents the component diagram for the CBC Digital Learning Platform, showing all internal components within each layer, their responsibilities, and the interfaces through which they communicate.

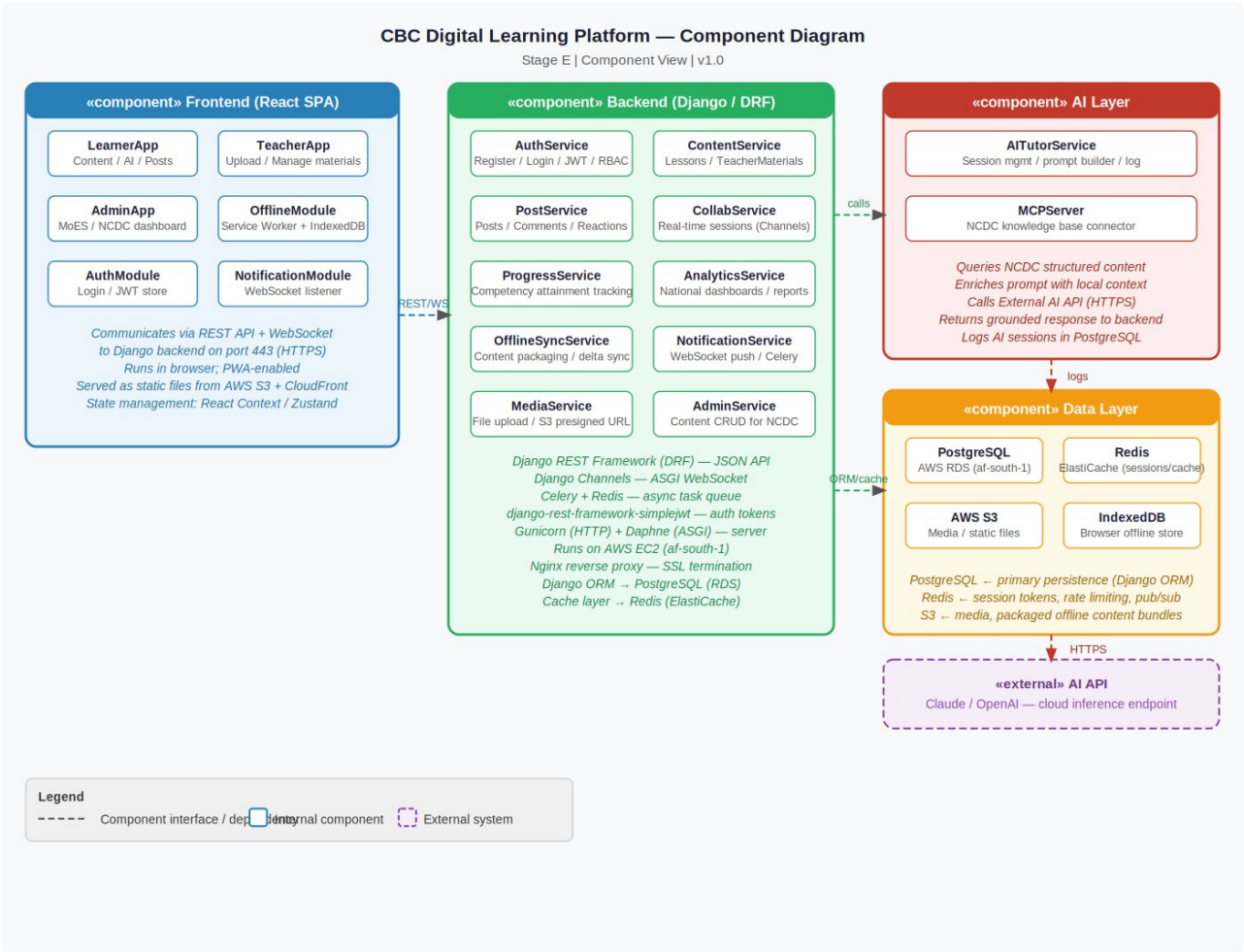

Figure E-02: CBC Digital Learning Platform — Component Diagram

3.2 Component Register

The table below formally registers each component, its layer, its primary responsibility, and its key SRS traceability reference.

Component	Layer	Responsibility	SRS Ref
LearnerApp	Presentation	Content browsing, AI tutor UI, post creation, collaboration	FR-L-03 to FR-L-11
TeacherApp	Presentation	Material upload, CBC tagging, content sharing	FR-T-01 to FR-T-04
AdminApp	Presentation	Curriculum management,	FR-M-01 to FR-

Component	Layer	Responsibility	SRS Ref
		national analytics dashboard	M-04
OfflineModule	Presentation	PWA service worker, IndexedDB sync, offline indicator	NFR-O-01 to NFR-O-03
AuthModule	Presentation	Login form, JWT token storage, session management	FR-L-01, FR-L-02
NotificationModule	Presentation	WebSocket listener, real-time event display	FR-L-08
AuthService	Application	User registration, login, bcrypt hashing, JWT, RBAC	NFR-SEC-02 to NFR-SEC-04
ContentService	Application	Lesson and TeacherMaterial CRUD, CBC tagging	FR-L-03, FR-T-02, FR-T-03
PostService	Application	Post creation, comments, reactions, visibility controls	FR-L-06, FR-L-07, FR-L-10
CollabService	Application	Real-time collaborative sessions via Django Channels	FR-L-08
ProgressService	Application	Competency attainment tracking and learner progress	FR-L-09
AnalyticsService	Application	National usage aggregation, report generation	FR-M-03, FR-M-04
OfflineSyncService	Application	Lesson packaging, delta sync, S3 bundle management	FR-L-04, NFR-O-01
MediaService	Application	File uploads, S3 presigned URL generation	FR-T-02
AdminService	Application	NCDC content CRUD, curriculum knowledge base management	FR-M-02
AITutorService	AI	Query routing, prompt building, AI session logging	FR-AI-01 to FR-AI-05
MCPServer	AI	Structured NCDC content retrieval for prompt enrichment	FR-AI-02, FR-AI-03
PostgreSQL (RDS)	Data	Primary relational store for all domain entities	NFR-SEC-01, NFR-SEC-02
Redis (ElastiCache)	Data	Sessions, JWT blacklist, Celery broker, WebSocket layer	NFR-SEC-03
AWS S3	Data	Binary media, static SPA build, offline bundles	FR-L-04
IndexedDB	Data	Browser-side offline lesson storage (PWA)	NFR-O-01, NFR-O-02

4. Deployment Diagram

4.1 Deployment Environment

The platform is deployed on Amazon Web Services in the af-south-1 region (Cape Town, South Africa). This region was selected as the closest AWS region to Uganda, minimising network latency for East African users compared to using European or US-based regions. All compute, database, and cache resources reside within a Virtual Private Cloud (VPC) with private subnets, ensuring no direct public internet access to backend infrastructure.

4.2 Deployment Diagram

Figure E-03 presents the full deployment view, showing all runtime nodes, the AWS services used, the network boundaries, and the communication paths between nodes.

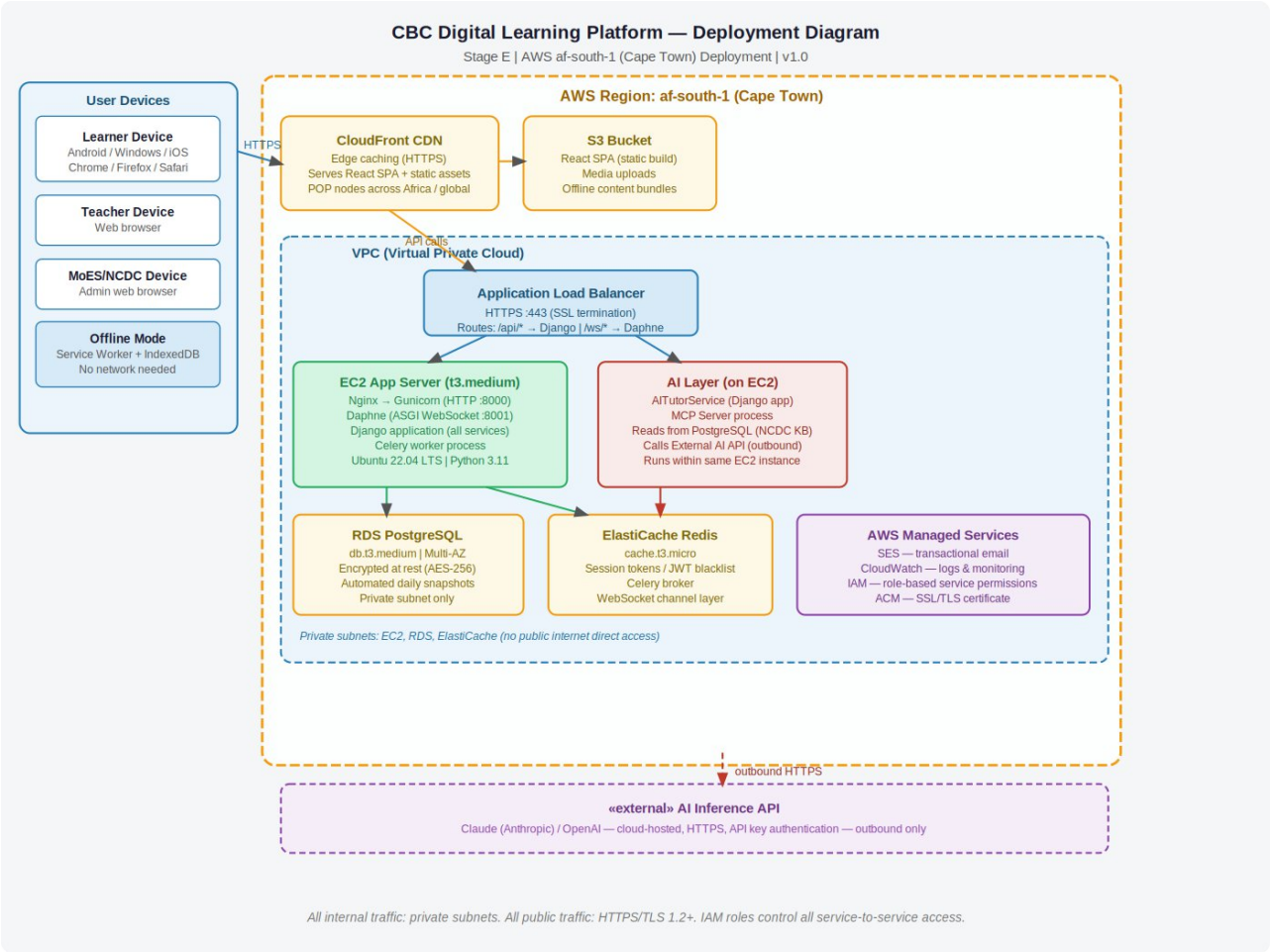

Figure E-03: CBC Digital Learning Platform — Deployment Diagram (AWS af-south-1)

4.3 Deployment Node Register

Node / Service	AWS Service	Role	Rationale
CloudFront CDN	AWS CloudFront	Serves React SPA and static assets via edge	Reduces latency; Africa/global POP coverage

Node / Service	AWS Service	Role	Rationale
Static SPA + Media	AWS S3	Hosts production React build and media uploads	Durable, cheap, scales without EC2
Application Load Balancer	AWS ALB	HTTPS termination, routes /api/ and /ws/ traffic	HA entry point; decouples DNS from EC2
App + AI Server	EC2 t3.medium	Runs Nginx, Gunicorn, Daphne, Django, Celery, MCP	Single instance for prototype; scalable later
Primary Database	RDS PostgreSQL	All domain data, NCDC knowledge base	Managed, encrypted, automated backups
Session / Cache / Broker	ElastiCache Redis	JWT tokens, Celery tasks, WebSocket channels	Managed Redis; removes single-point of failure
Email Service	AWS SES	Transactional notifications (future phase)	Deliverable, low cost, Uganda region routable
Monitoring & Logs	AWS CloudWatch	Log aggregation, uptime alerts, error tracking	Required for NFR-A-01 99% uptime target
TLS Certificate	AWS ACM	SSL/TLS cert for all HTTPS traffic	Auto-renewed; satisfies NFR-SEC-01
IAM Roles	AWS IAM	Service-to-service permissions (EC2, S3, RDS, SES)	Principle of least privilege
External AI API	Claude / OpenAI (external)	LLM inference for AI tutor responses	Managed inference; no self-hosting cost

5. API Contracts

5.1 API Design Principles

The platform exposes a RESTful JSON API served by Django REST Framework on all routes under /api/v1/. Authentication uses JWT Bearer tokens issued by the AuthService. All endpoints enforce RBAC: each route declares which roles (learner, teacher, moES_official) are permitted. WebSocket connections for real-time collaboration and notifications are served by Django Channels under the /ws/ path prefix.

API versioning is path-based (/api/v1/) to allow future API evolution without breaking existing clients. All responses follow a consistent envelope: { "status": "success"|"error", "data": {...}, "message": "..." }.

5.2 Authentication Endpoints

Endpoint	Method	Role	Description	SRS Ref
/api/v1/auth/register/	POST	Public	Register new learner or teacher account	FR-L-01, FR-T-01
/api/v1/auth/login/	POST	Public	Authenticate user; return JWT access + refresh tokens	FR-L-02, FR-T-01
/api/v1/auth/logout/	POST	All roles	Blacklist refresh token; invalidate session	FR-L-02
/api/v1/auth/token/refresh/	POST	All roles	Issue new access token using valid refresh token	FR-L-02
/api/v1/auth/me/	GET	All roles	Return authenticated user profile	FR-L-01

5.3 Content Endpoints

Endpoint	Method	Role	Description	SRS Ref
/api/v1/content/lessons/	GET	Learner, Teacher	List lessons filtered by subject, level, competency	FR-L-03
/api/v1/content/lessons/{id}/	GET	Learner, Teacher	Retrieve full lesson content for display or offline cache	FR-L-03, FR-L-04
/api/v1/content/lessons/	POST	MoES Official	Create new CBC lesson (NCDC-authored)	FR-M-02
/api/v1/content/materials/	GET	Learner, Teacher	List teacher-uploaded materials by class/subject	FR-T-02, FR-T-04
/api/v1/content/materials/	POST	Teacher	Upload new	FR-T-02, FR-

Endpoint	Method	Role	Description	SRS Ref
			teaching material with CBC tags	T-03
/api/v1/content/offline-bundle/	GET	Learner	Fetch packaged lesson bundle for offline download	FR-L-04, NFR-O-01
/api/v1/content/subjects/	GET	All roles	List all CBC subjects	FR-L-03
/api/v1/content/competencies/	GET	All roles	List competencies by subject and class level	FR-L-09

5.4 AI Tutor Endpoints

Endpoint	Method	Role	Description	SRS Ref
/api/v1/ai/query/	POST	Learner	Submit academic question; returns AI-grounded response	FR-L-05, FR-AI-01
/api/v1/ai/sessions/	GET	Learner	List learner's AI session history (private by default)	FR-AI-01, NFR-SEC-05
/api/v1/ai/sessions/{id}/	GET	Learner	Retrieve a specific AI session transcript	FR-L-10

5.5 Posts and Collaboration Endpoints

Endpoint	Method	Role	Description	SRS Ref
/api/v1/posts/	GET	Learner, Teacher	List posts visible to requesting user	FR-L-07
/api/v1/posts/	POST	Learner, Teacher	Create a new post with visibility setting	FR-L-06, FR-L-10
/api/v1/posts/{id}/	GET / PATCH / DELETE	Author / roles	Retrieve, update, or delete a post	FR-L-06, FR-L-11
/api/v1/posts/{id}/comments/	GET / POST	Learner	List or add comments to a post	FR-L-07
/api/v1/posts/{id}/react/	POST	Learner	Add LIKE or SHARE reaction to a post	FR-L-07
/ws/collab/{session_id}/	WebSocket	Learner	Real-time collaborative learning session channel	FR-L-08

5.6 Progress and Analytics Endpoints

Endpoint	Method	Role	Description	SRS Ref
/api/v1/progress/me/	GET	Learner	Retrieve learner's own competency attainment record	FR-L-09
/api/v1/progress/{id}/update/	POST	System (internal)	Update competency progress after lesson completion	FR-L-09
/api/v1/analytics/national/	GET	MoES Official	National dashboard: usage, attainment by region/school	FR-M-03
/api/v1/analytics/reports/	GET / POST	MoES Official	Generate and export national learning reports	FR-M-04

6. Key Architectural Decisions

The table below records the major architectural decisions made in Stage E, the alternatives considered, and the rationale for each decision. This record ensures full traceability from the architecture back to stakeholder requirements.

Decision	Choice Made	Alternative Considered	Rationale
Architecture style	Layered Monolith	Microservices	Appropriate for prototype scope; lower operational complexity; layer structure enables future extraction
Backend framework	Django + DRF	Node.js / FastAPI	Team Django expertise; mature ORM; DRF provides clean serialiser + auth patterns; Channels for WebSocket
Frontend framework	React SPA	Next.js SSR	No SSR needed for a web app; SPA delivers offline PWA capability cleanly; simpler deployment via S3+CloudFront
AI integration	External API + MCP Server	Self-hosted LLM	Prototype budget; production-grade inference quality; MCP pattern grounds responses in NCDC content per FR-AI-03
Deployment region	AWS af-south-1 (Cape Town)	EU or US region	Closest AWS region to Uganda; lowest latency for East African users; data sovereignty within Africa
Offline strategy	PWA (Service Worker + IndexedDB)	Native mobile app	Web-only constraint per SRS; PWA delivers offline within browser; no app store required
Async tasks	Celery + Redis	Django background threads	Robust task queue; Redis dual-use (cache + broker); required for analytics aggregation at national scale
Real-time	Django Channels (ASGI)	Polling / SSE	Native WebSocket support; integrates with Django auth and ORM; scales with Redis channel layer
Database	PostgreSQL (AWS RDS)	MySQL / MongoDB	Relational integrity for CBC structure; RDS managed service; JSONB for flexible AI session logs if needed

7. Architecture Traceability to SRS

This section confirms that every architectural element introduced in Stage E traces directly to a requirement in the SRS v1.0 and to a design decision in the stages preceding it.

SRS Requirement	Addressed By (Stage E Element)
FR-L-01, FR-L-02 — Learner registration/login	AuthService (Application Layer), AuthModule (Presentation Layer), JWT + bcrypt
FR-L-03, FR-L-04 — CBC content access and offline	ContentService, OfflineSyncService, OfflineModule (PWA), AWS S3 bundles, IndexedDB
FR-L-05, FR-AI-01 to FR-AI-05 — AI Tutor	AITutorService, MCPServer, External AI API, /api/v1/ai/ endpoints
FR-L-06, FR-L-07 — Posts and peer sharing	PostService, LearnerApp, /api/v1/posts/ endpoints
FR-L-08 — Real-time collaboration	CollabService, Django Channels, /ws/collab/ WebSocket, Redis channel layer
FR-L-09 — Competency progress tracking	ProgressService, /api/v1/progress/ endpoints, PostgreSQL CompetencyProgress records
FR-T-01 to FR-T-04 — Teacher content management	ContentService, MediaService, TeacherApp, /api/v1/content/materials/ endpoints
FR-M-01 to FR-M-04 — MoES/NCDC governance	AdminService, AnalyticsService, AdminApp, /api/v1/analytics/ endpoints
NFR-P-01, NFR-P-03 — Low-bandwidth performance	CloudFront CDN, S3 compression, lazy loading in React, progressive image delivery
NFR-S-01, NFR-S-02 — 100k concurrent scale	ALB horizontal scaling, RDS Multi-AZ, ElastiCache Redis, EC2 auto-scaling group (future)
NFR-A-01 — 99% uptime	AWS ALB + RDS Multi-AZ HA, CloudWatch alerting, graceful AI layer degradation
NFR-SEC-01 to NFR-SEC-06 — Security and privacy	HTTPS/TLS via ACM, bcrypt auth, JWT RBAC, private subnets, IAM roles, RDS encryption at rest
NFR-O-01 to NFR-O-03 — Offline support	PWA Service Worker, IndexedDB, OfflineSyncService, offline indicator in OfflineModule
NFR-AC-01 to NFR-AC-05 — Accessibility	React SPA responsive design, CloudFront delivery, mobile-first CSS, language i18n architecture

8. Document Sign-off

This Software Architecture Document (SAD) represents the agreed architectural baseline for the CBC Digital Learning Platform following completion of Stage E of the Unified Software Architectural Process. All subsequent stages — Stage F (Database & Data Design), Stage G (UI/UX Design), and Stage H (Architecture Validation & Review) — shall be conducted in conformance with the architecture defined in this document.

Modifications to this architecture after sign-off require a formal change request and must be assessed for impact on all dependent stages before being incorporated.

Role	Name	Signature / Date
Project Supervisor	_____	_____
Software Architect / Group Leader	_____	_____
NCDC / Stakeholder Representative	_____	_____